

SMARTHOME GESTURE CONTROL APPLICATION PROJECT PART 2 REPORT

1. Introduction

The SmartHome Gesture Control Application aims to recognize hand gestures for controlling smart home devices. This project utilizes a pre-trained **Convolutional Neural Network (CNN) model** to extract feature vectors from gesture videos. The extracted features from the **penultimate layer** of the CNN model are used to classify gestures based on **cosine similarity**. The system processes **training and test gesture videos**, extracts relevant features, and identifies gestures accurately.

2. Approach to the Problem

The implementation follows a structured approach:

1. **Extract middle frame from training videos** – Each training gesture video is processed, and the middle frame is extracted.
2. **Extract features from the penultimate layer** – The extracted frames are passed through the CNN model to obtain feature vectors from the second-to-last layer.
3. **Store training gesture features** – The extracted feature vectors are stored for future comparison.
4. **Process test gesture videos** – Middle frames of test videos are extracted and passed through the CNN model to obtain feature vectors.
5. **Compare test features with training features using cosine similarity** – The test gesture feature vectors are compared to training gesture vectors, and the closest match is assigned as the recognized gesture.
6. **Save results to Results.csv** – The recognized gesture labels are stored in a CSV file in a **51x1 matrix format** without headers.

3. Solution Implementation

Feature Extraction from Training and Test Videos

- The **process_video(video_path)** function extracts the middle frame from each video.
- The **extract_features(frame)** function utilizes a pre-trained CNN model to extract a feature vector from the **penultimate layer**.
- Extracted **feature vectors** are stored in a dictionary (training_features).

Gesture Recognition using Cosine Similarity

- Each test **gesture feature vector** is compared against all **training gesture feature vectors** using **cosine similarity**.
- The **training gesture with the lowest cosine similarity score** is selected as the recognized gesture.
- The **recognized gesture** is stored in **Results.csv**.

4. Challenges and Solutions

Challenge 1: Extracting Relevant Features

- The project required extracting the **penultimate layer** of the CNN model instead of the final classification layer.
- **Solution:** Implemented `tf.keras.Model(model.input, model.layers[-2].output)` to correctly extract features.

Challenge 2: Handling Empty or Corrupted Test Videos

- Some test videos might not contain valid frames.
- **Solution:** Implemented **error handling** in `process_video()` to **skip invalid frames** and prevent program crashes.

Challenge 3: Ensuring Correct Output Format for Gradescope

- Gradescope requires a **51x1 matrix format** without headers.
- **Solution:** Modified the **CSV generation** code to ensure only **integer gesture labels** are written without headers.

5. Improvements and Future Enhancements

Improvements Implemented

- ✓ **Middle Frame Extraction:** Improved accuracy by selecting the **middle frame** instead of the first frame.
- ✓ **Error Handling:** Implemented **checks** to prevent errors when processing **empty or corrupted** videos.
- ✓ **Optimized Feature Extraction:** Extracted **penultimate layer features** instead of using **raw frames** for better recognition.

Future Enhancements

- ◆ **Use a More Advanced Similarity Measure:** Explore **Euclidean distance** or a trainable classifier for improved accuracy.
- ◆ **Improve Gesture Dataset:** Collect a **larger and more diverse dataset** to enhance **model generalization**.
- ◆ **Real-Time Gesture Recognition:** Extend the application to work **in real-time** using **live video feeds**.

6. Conclusion

The **SmartHome Gesture Control Application** successfully processes **gesture videos**, extracts **meaningful features**, and **classifies gestures** using **cosine similarity**. The project follows the required workflow, and the final output (**Results.csv**) meets **grading requirements**. Future improvements can further **enhance accuracy** and extend **functionality** to **real-time applications**.